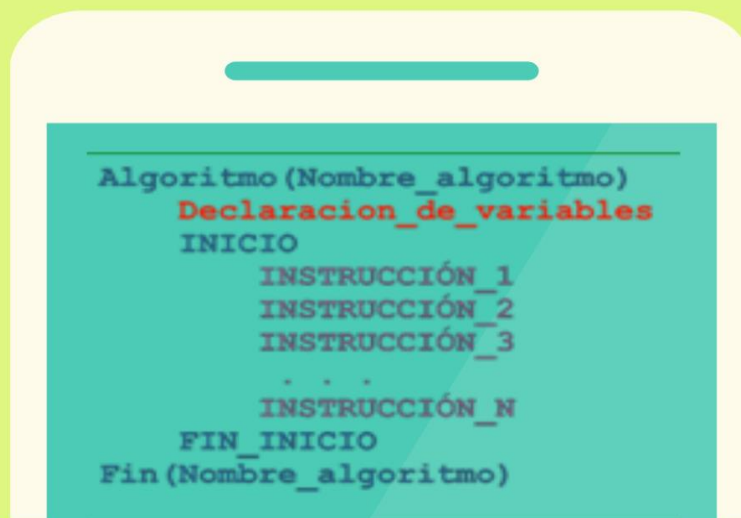


MATERIAL DE ESTUDIO

ALGORITMOS Y ESTRUCTURAS DE DATOS

*Ingeniería en Sistemas de Información
UTN Facultad Regional Resistencia*



CONCEPTOS Y PROBLEMAS

ALGORITMOS Y ESTRUCTURAS DE DATOS

Subacciones

Tabla de contenido

OBJETIVOS	3
CONTENIDOS	3
SUBACCIONES.....	4
Control de Ejecución	5
Elementos de las Subacciones	5
Funciones	Error! Bookmark not defined.
Declaración de Funciones	7
Procedimientos	Error! Bookmark not defined.

ALGORITMOS Y ESTRUCTURAS DE DATOS

Subacciones

OBJETIVOS

- a. Identificar las acciones involucradas en un problema.
- b. Reconocer los estados de los objetos antes y después de la acción realizada.
- c. Descomponer una acción en términos de otras más elementales.
- d. Utilizar un léxico mínimo común.
- e. Comprender el significado del término Algoritmo

CONTENIDOS

Subacciones

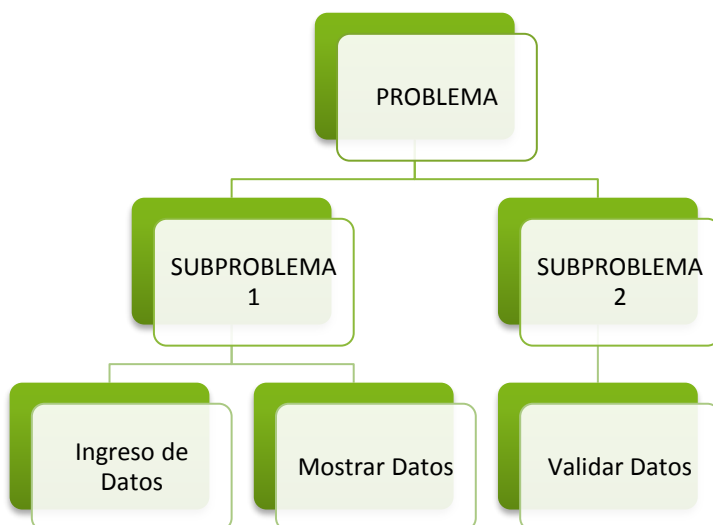
- Funciones
- Procedimientos
- Paso de parámetros

ALGORITMOS Y ESTRUCTURAS DE DATOS

Subacciones

SUBACCIONES

La resolución de problemas complejos se facilita considerablemente si se dividen en problemas más pequeños; y la resolución de estos subproblemas se realiza mediante **subalgoritmos** o **subacciones**.



Subacciones o Subalgoritmos

- Son acciones que **forman parte** de una ACCIÓN PRINCIPAL.
- Son módulos que están escritos para ejecutar alguna **tarea específica**.
- Se definen en el **AMBIENTE** pues van a ser utilizadas e invocadas durante el **PROCESO** del algoritmo diseñado
- Se **escriben solamente una vez**, pero pueden ser referenciados en diferentes puntos del algoritmo, de modo que se puede evitar la duplicación innecesaria del código.

ALGORITMOS Y ESTRUCTURAS DE DATOS

Subacciones

Control de Ejecución

El algoritmo principal se ejecuta en una primera instancia, y da la orden de inicio de ejecución de los subalgoritmos. Esto puede suceder N veces.

Cuando realiza la solicitud, el algoritmo se detiene hasta que el subalgoritmo deja de realizar su tarea, luego continúa; esto se conoce como control de ejecución.

Elementos de las Subacciones

a) Nombre

Las subacciones que veremos en nuestra materia se definen siempre utilizando un *Nombre*.

Este *Nombre* debe ser único, representativo del objetivo específico del subalgoritmo que identifica y no debe ser una palabra reservada (por ejemplo Escribir, Leer....).

b) Parámetros

En algunos casos las subacciones necesitarán pasar o recibir datos para ejecutarse y allí hacemos uso de parámetros.

Los **parámetros** son *variables y/o constantes* para pasar datos entre algoritmos y subalgoritmos en ambos sentidos.

- Los parámetros que aparecen en la definición del subalgoritmo, se denominan **parámetros formales o ficticios**.
- Los parámetros que se encuentran en la llamada al subalgoritmo se denominan **parámetros actuales o argumentos**. Son los datos que el algoritmo principal envía a la subacción. Estos parámetros contienen los valores necesarios para evaluar el subalgoritmo en ese momento.

Los parámetros actuales pueden ser constantes, variables, expresiones. En cambio, los parámetros formales sólo pueden ser variables.

Cada vez que un algoritmo llama a una subacción se establece una correspondencia entre los parámetros actuales y los parámetros formales, verificando que las cantidades sean iguales y que los tipos de datos sean los esperados de acuerdo con el orden definido. Veremos ejemplos posteriores para que puedan comprender esta cuestión.

ALGORITMOS Y ESTRUCTURAS DE DATOS

Subacciones

FUNCIONES

Una **FUNCIÓN** es un subalgoritmo que recibe, como ***argumentos*** o ***parámetros actuales***, datos de tipo numérico o no numérico, y **devuelve un único resultado**.

FUNCIONES INTERNAS

- *Las funciones incorporadas al sistema se denominan funciones internas, o intrínsecas predefinidas*

FUNCIONES EXTERNAS

- *Las funciones definidas por el usuario se llaman funciones externas*



Podés buscar ejemplos de funciones predefinidas en lenguajes de programación, por ejemplo PASCAL.

ALGORITMOS Y ESTRUCTURAS DE DATOS

Subacciones

Declaración de Funciones

En pseudocódigo usaremos la siguiente sintaxis.

```
Funcion nombrefun(lista de parámetros): Tipo
```

```
..
```

```
...(Declaraciones locales si fuera necesario > Ambiente)
```

```
..
```

```
..(Acciones de la funcion)
```

```
...
```

```
nombrefun:= <valor de la función>
```

```
Fin Funcion
```

No lo olvides!

Debe estar para
que la función
devuelva un valor

nombrefun : es el nombre de la función.

lista de parámetros : es la lista de parámetros formales. **Esta lista NO puede ser vacía.**

Tipo: es el tipo de resultado que devuelve la función

El algoritmo principal invoca la función con el nombre de esta última en una expresión seguida de una lista de argumentos que deben coincidir en cantidad, tipo y orden con los de la función que fue definida.

*Para que las acciones descritas en una **FUNCIÓN** sean ejecutadas, se necesita que éste sea invocado desde un algoritmo principal o desde otros subalgoritmos a fin de proporcionarle los argumentos de entrada necesarios para realizar estas acciones.*



ALGORITMOS Y ESTRUCTURAS DE DATOS

Subacciones

Ejemplo

FUNCION **ES_PRIMO** (A:entero):lógico

AMBIENTE

i: entero

PROCESO

ES_PRIMO:=V //habilito la función como bandera

PARA i:=2 hasta (A-1) **HACER**

SI A mod i = 0 **ENTONCES**

ES_PRIMO:=F //ya no es primo

FIN_SI

FIN_PARA

FIN_FUNCION

ALGORITMOS Y ESTRUCTURAS DE DATOS

Subacciones

PROCEDIMIENTOS

Un **PROCEDIMIENTO** también es una subacción, diseñado con un fin específico, pero a diferencia de las funciones no devuelve ningún valor, solo ejecuta acciones.

Veamos un ejemplo

Desarrollar un procedimiento que muestre por pantalla la validación del login de usuario

PROCEDIMIENTO Login(usu,pass,valid:alfanumerico)

Si pass = valid entonces

ESC('Acceso autorizado a ', usu)

Sino

Esc('Login inválido para ', usu)

FinSi

FIN_PROCEDIMIENTO

Por qué usar procedimientos?

La estrategia de usar procedimientos al diseñar soluciones algorítmicas, y luego más adelante tus programas, se basa en las siguientes ventajas:

- Utilizar procedimientos **facilitan el diseño top-down**¹ de las soluciones.
- Se pueden **ejecutar más de una vez** en un algoritmo, con solo realizar su definición en el ambiente y llamarlos las veces que se requieran
- Su uso **facilita la división de tareas** entre los desarrolladores de un equipo. Entonces si debes resolver un problema complejo pueden dividirse tareas y generar procedimientos para luego integrarlos en un único algoritmo.

¹ Te recomendamos que busques qué significa “*diseño top-down*” en nuestro ámbito.

ALGORITMOS Y ESTRUCTURAS DE DATOS

Subacciones

ÁMBITOS DE VARIABLES

Usaremos el término **ámbito de las variables** para referirnos a aquel conjunto de partes del algoritmo en el que una variable es conocida. Así distinguiremos:

Variables Locales

Como vimos las funciones y procedimientos permiten modularizar un algoritmo.

Todas las variables declaradas dentro de una subacción son **variables locales** (es decir, son conocidas solo en la función o procedimiento en la cual han sido definidas), por lo tanto ninguna otra subacción tiene acceso a ellas.

La mayor parte de las subacciones tienen una lista de parámetros. Los parámetros de una subacción también son variables locales.

Las variables locales comienzan su existencia cuando la subacción es llamada por el algoritmo principal y desaparecen cuando esta subacción termina su ejecución, por esta razón las variables locales no retienen sus valores de una llamada a otra.

```
13 FUNCION EsMultiploDe10 (numero:entero):logico
14     SI numero MOD 10 = 0 ENTONCES
15         EsMultiploDe10 := V
16     CONTRARIO
17         EsMultiploDe10 := F
18     FIN_SI
19 FIN_FUNCION
20
```

```
9     FIN_SI
10     FIN_PARA
11 FIN_FUNCION
12
13 ...
14 Leer(codigo)
15 bandera:=EsMultiploDe10(codigo)
16 SI bandera ENTONCES
17     ....
18
```

Variables Globales

Las variables declaradas en el algoritmo principal se llaman **variables globales**, pues pueden ser utilizadas en el algoritmo principal y en todas las subacciones en él declaradas.

ALGORITMOS Y ESTRUCTURAS DE DATOS

Subacciones

PASO DE PARAMETROS

El **paso de parámetros** es el proceso por el cual el algoritmo principal envía variables a una función o procedimiento, de forma tal que estas subacciones reciban datos para trabajarlos internamente.

Existen dos alternativas:

- Paso de parámetros por *valor*
- Paso de parámetros por *referencia*

En la cátedra vamos a utilizar, en mayor parte, el paso de parámetro por valor.

Paso de parámetro por valor

Cuando surge la necesidad de obtener el valor o contenido de una variable local desde el algoritmo principal hacia una subacción, se está utilizando la estrategia de **paso de parámetros por valor**.

En este caso se produce una copia de la variable original hacia el argumento formal de la función o procedimiento que lo recibe. Dicha variable tendrá como ámbito la función receptora o procedimiento receptor y al culminar su ejecución se liberará el espacio de memoria que ocupa.

Si usamos parámetros por valor, nunca podremos modificar los valores de las variables originales ya que se producen copias de las mismas hacia las variables de la función llamada y su ámbito es local.

